School of Future Tech

Case Study Report

On

Movie Trivia Battle Website

by

Pari Gothi

150096725097

Index

1. Introduction to the Case Study.
2. Problem Statement / Case Background (Abstract).
3. Problem Statement / Case Study Design.
4. Methods & Algorithms Technology Applied in the Problem Statement / Case Study.
5. Problem Statement / Case Study Implementation Details and Snapshots.
6. Problem Statement / Case Study Results and Conclusion.
7. References

1. Introduction to the Case Study

Online trivia and quiz games are widely enjoyed for entertainment, learning, and competitive engagement. However, many existing platforms are static, single-player, and lack dynamic scoring or genre-based challenges. This case study focuses on designing and implementing a Movie Trivia Battle Website is an interactive, multi-genre trivia game where users answer timed questions, earn points, and compete through a persistent leaderboard.

The project demonstrates how front-end web technologies (HTML, CSS, and JavaScript) can be combined to build a fully functional browser-based game. It highlights the role of DOM manipulation, timer logic, local storage, and responsive UI design in delivering an engaging user experience across multiple pages.

2. Problem Statement / Case Background (Abstract)

Background

Most browser-based trivia games offer a generic, unsegmented question pool and do not reward speed or accuracy. Players have no way to track performance over time, choose preferred genres, or engage with other users. A genre-specific, timed, and leaderboard-driven trivia system better serves competitive and casual players alike.

Abstract

This case study presents the design and implementation of a Movie Trivia Battle Website built using HTML5, CSS3, and vanilla JavaScript. The application features six movie genres (Action, Comedy, Sci-Fi, Horror, Romance, Classic), each containing a bank of ten questions. During gameplay, a 15-second countdown timer is displayed per question, and bonus points are awarded for fast correct answers. A scoring algorithm calculates results based on both accuracy and response time. Final scores, names, genre, accuracy percentage, and date are stored in localStorage and rendered in a sortable leaderboard. A community forum page allows players to post tips and hot takes using browser-persisted data. The system is organized across four HTML pages — Home, Play, Leaderboard, and Forum — connected by a shared stylesheet and JavaScript file.

3. Case Study Design

The Movie Trivia Battle Website is designed as a multi-page application with the following stages:

1. Home Page (index.html)
   Introduces the game with branding, genre tags, feature highlights (timed rounds, question count, rank tiers), and a Play Now call-to-action button.
2. Game Setup & Battle (play.html)
   Player enters their name and selects a genre. The game loop displays shuffled questions one at a time with a 15-second timer, four multiple-choice options, and live score tracking. Correct/wrong feedback is displayed before advancing to the next question.
3. Results & Score Saving

After 10 questions, a results screen shows total score, correct/wrong counts, accuracy percentage, and a rank badge (Rookie to Cinephile). The player can save their score to localStorage.

4. Leaderboard (leaderboard.html)
   Retrieves and displays all saved scores from localStorage in a ranked table, showing position, name, score, genre, accuracy, and date.
5. Forum (forum.html)
   A community posting board pre-loaded with default posts. Users can add their name and a message, which is stored in and retrieved from localStorage.

4. Methods & Algorithms Technology Applied in the Problem Statement / Case Study

Key methods and algorithms:

1. Game Logic Methods

- Fisher-Yates-style shuffle (sort with Math.random) to randomize question order each session.
- Timer countdown using setInterval, clearing on answer selection or time expiry.
- Dynamic scoring: base 100 points per correct answer plus time bonus (Math.ceil(time * 6.67)).

2. DOM Manipulation

- innerHTML and createElement used to dynamically render questions, choices, leaderboard rows, and forum posts.
- CSS class toggling (.correct, .wrong, .show, .active) for visual feedback and page navigation.

3. Data Persistence

- localStorage used to store and retrieve JSON-serialized score entries and forum posts across sessions.
- Scores sorted by descending score value and capped at 50 entries.

4. Ranking Algorithm

- Threshold-based rank assignment: score >= 1500 = CINEPHILE, >= 1200 = FILM BUFF, >= 900 = CRITIC, >= 600 = ENTHUSIAST, else ROOKIE.

Technology Stack Used for the Case

- Markup language: HTML5 for semantic page structure and multi-page layout.
- Styling: CSS3 with custom properties, flexbox, grid, media queries, and CSS transitions.
- Scripting: Vanilla JavaScript (ES6+) for all game logic, event handling, and local storage operations.
- Storage: Browser localStorage API for persisting scores and forum posts without a backend.
- Environment: Any modern browser (Chrome, Firefox, Edge, Safari). No build tools or frameworks required.

- Design: Custom cinematic UI theme with a warm gradient background, dark navigation bar, and responsive layout.

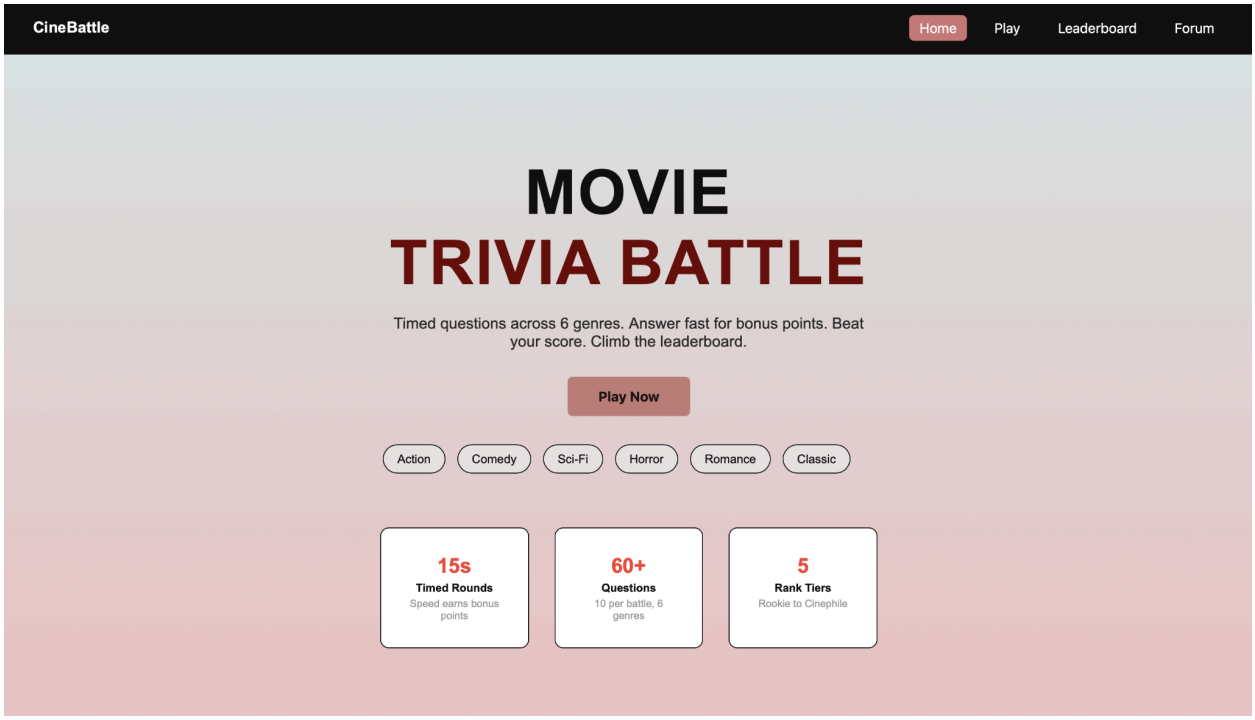
5. Problem Statement / Case Study Implementation Details and Snapshots.

The implementation is organized across four HTML files, one shared CSS file, and one JavaScript file:

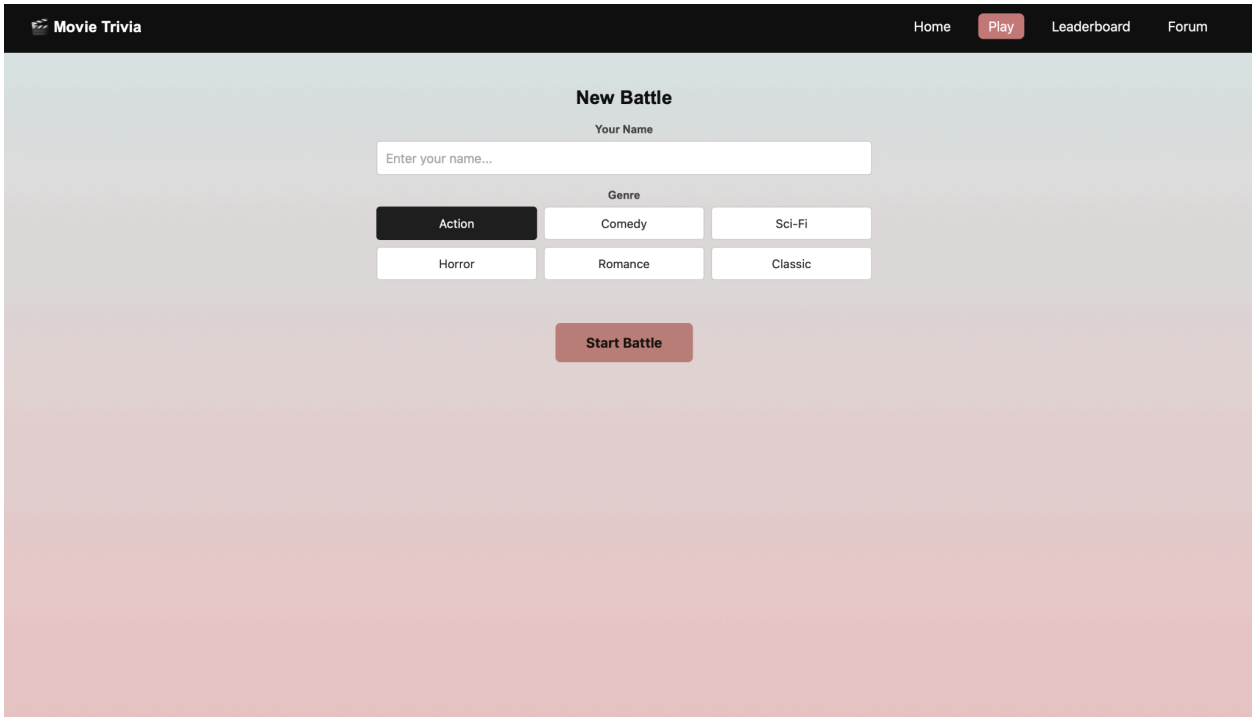
1. index.html — Home Page
   Displays the CineBattle brand, genre tag pills, feature statistics (15s timed rounds, 60+ questions, 5 rank tiers), and a Play Now button linking to play.html.
2. play.html — Setup, Battle, and Results Screens
   Three sub-sections toggled by display style: the setup form (name + genre grid), the live battle view (info bar, timer bar, question box, and 2x2 choice grid), and the results screen (badge, score, stats, and save/replay buttons).
3. leaderboard.html — Score Table
   An HTML table populated by renderLB() on page load, reading from localStorage. Medal emojis awarded for the top three positions. Includes a Clear Scores button.
4. forum.html — Community Forum
   A compose box (textarea + name input) and a dynamically rendered post list. On first load, three default posts are shown. Users can add new posts which are prepended and saved.
5. style.css — Shared Styles
   Global reset, body gradient, nav bar, page containers, button styles, genre grid, battle UI (timer bar, question box, choice buttons), results layout, leaderboard table, forum post cards, toast notifications, and a responsive media query for screens under 500px.
6. script.js — All Game Logic
   Question bank object (Q) with 10 questions per genre, state variables, startGame(), nextQ(), pick(), timesUp(), endGame(), saveScore(), renderLB(), clearScores(), addPost(), renderPosts(), and toast() utility.

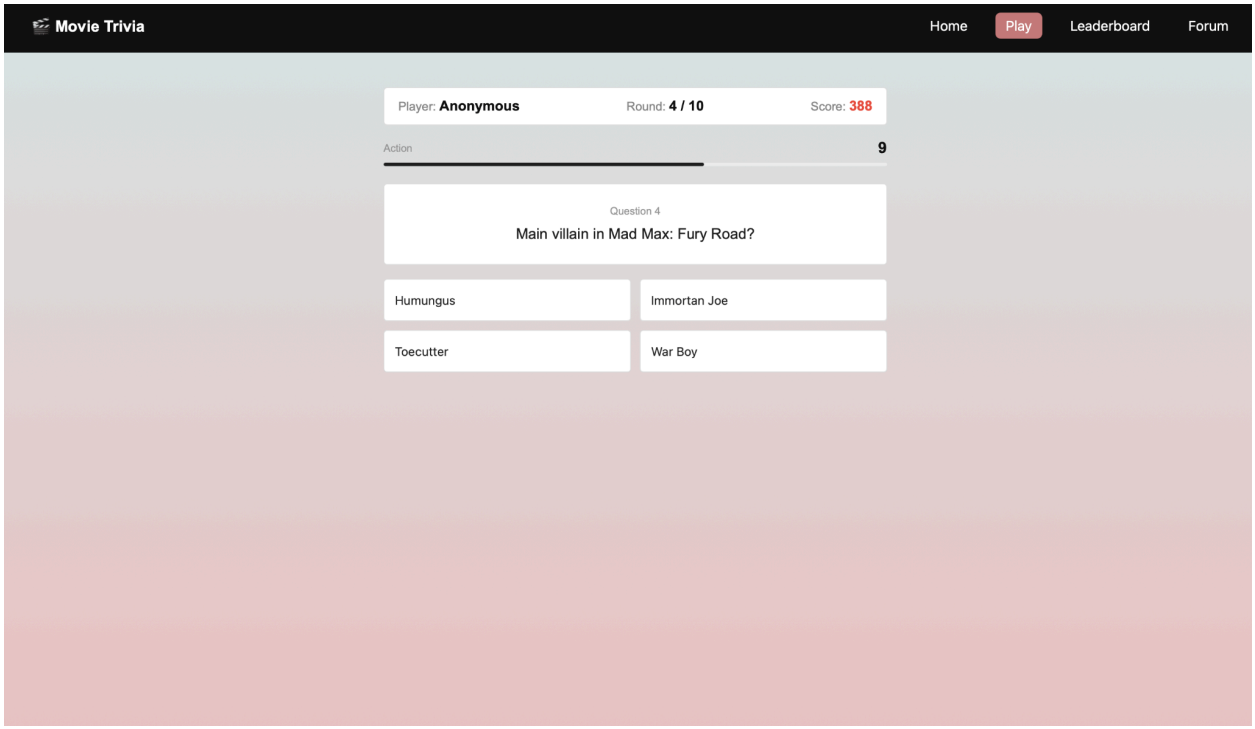
Snapshots:

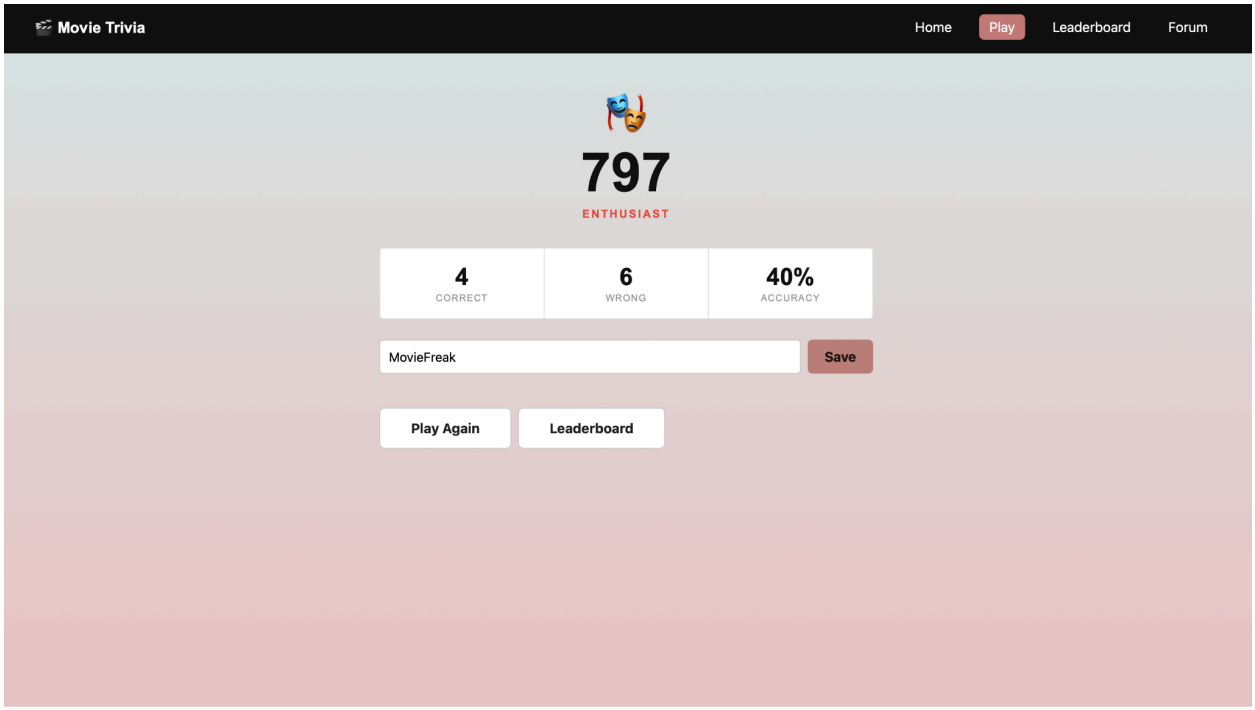
1. Landing Page:


CineBattle
Home
Play
Leaderboard
Forum
MOVIE
TRIVIA BATTLE
Timed questions across 6 genres. Answer fast for bonus points. Beat your score. Climb the leaderboard.
Play Now
Action
Comedy
Sci-Fi
Horror
Romance
Classic
15s
Timed Rounds
Speed earns bonus points
60+
Questions
10 per battle, 6 genres
5
Rank Tiers
Rookie to Cinephile

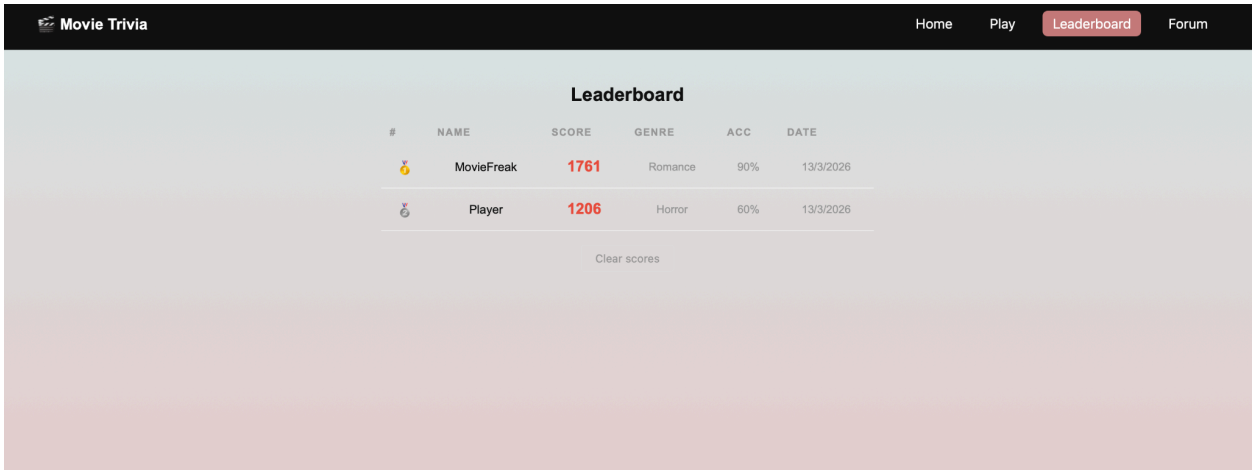
2. Play:


Movie Trivia
Home
Play
Leaderboard
Forum
New Battle
Your Name
Enter your name...
Genre
Action
Comedy
Sci-Fi
Horror
Romance
Classic
Start Battle

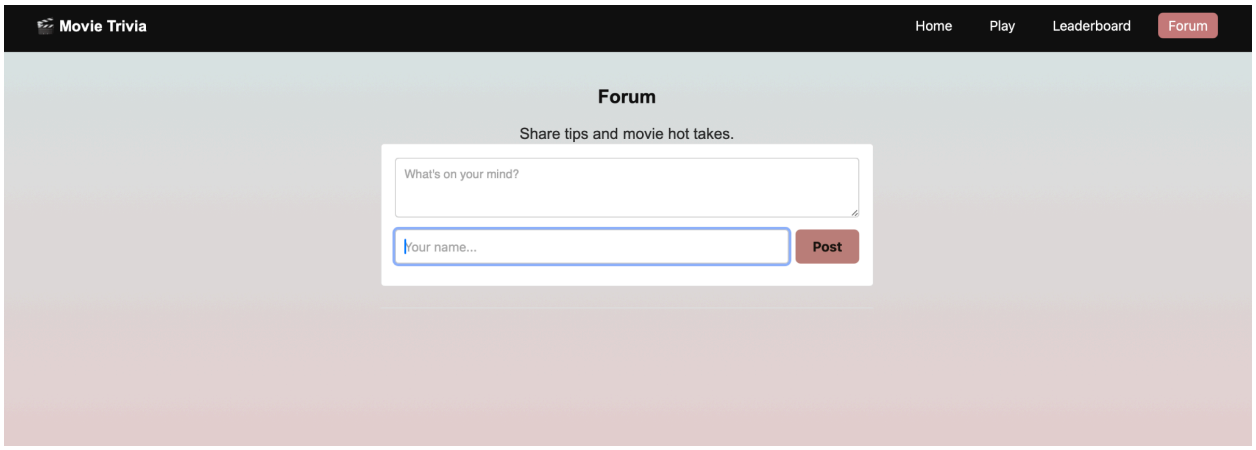




3. Leaderboard:



4. Forum:



6. Problem Statement / Case Study Results and Conclusion.

Findings

- A 15-second timer with a dynamic score bonus incentivizes faster, confident answers and increases competitive replay value.
- Shuffling questions each session from a 10-question genre pool prevents memorization and keeps gameplay fresh.
- localStorage provides zero-backend persistence sufficient for a browser-based single-player leaderboard and forum.
- Rank tiers (Rookie to Cinephile) combined with emoji badges deliver satisfying feedback and motivate improvement.
- Sharing one stylesheet and script across all four pages keeps the codebase maintainable and ensures a consistent visual identity.

Conclusion

This case study demonstrates a complete, deployment-ready Movie Trivia Battle game built entirely with vanilla HTML, CSS, and JavaScript. It integrates timed game mechanics, dynamic score calculation, genre selection, persistent leaderboard tracking, and a community forum into a

cohesive multi-page web application, providing a strong foundation for further enhancement such as backend integration, multiplayer modes, or additional question packs.

7. References

- MDN Web Docs — HTML5, CSS3, and JavaScript language references
- CSS-Tricks — Flexbox and CSS Grid layout guides.
- JavaScript.info — In-depth JavaScript concepts including closures, timers, and DOM manipulation.